

Real-Time Traffic Anomaly Detection Project Report

1. Project Overview

This project is an end-to-end MLOps system for detecting anomalies in Estonian traffic census data. It processes hourly traffic counter measurements, engineers traffic and time-based features, trains an unsupervised anomaly detection model, exposes predictions through an API, visualizes results in a dashboard, and monitors feature drift over time.

The main use case is to identify unusual traffic patterns such as sudden volume drops, unexpected congestion, unusual speed behavior, or sensor-related anomalies.

2. Data

The raw dataset is Estonian Transport Administration traffic census data. The repository contains yearly CSV files from 2018 through 2026:

- data/ll_2018.csv
- data/ll_2019.csv
- data/ll_2020.csv
- data/ll_2021.csv
- data/ll_2022.csv
- data/ll_2023.csv
- data/ll_2024.csv
- data/ll_2025.csv
- data/ll_2026.csv

Each row represents one traffic counter, one lane, and one hour. Important raw fields include:

- id: traffic counter identifier
- kanal: lane identifier
- aeg: timestamp
- 1 to 10: vehicle type counts
- <40Kph to =>130: speed-bin counts

The raw CSV files are tracked with DVC .dvc pointer files, and the configured DVC remote is an S3-compatible Backblaze B2 bucket:

```
text s3://mlops-project/dvc https://s3.eu-central-003.backblazeb2.com
```

The main processed dataset is:

```
text data/processed/traffic_data_processed.parquet
```

This parquet file is produced by the preprocessing stage and is used by the dashboard, monitoring, and DVC pipeline dependency graph.

3. Architecture

The project is organized as a layered MLOps application:

Step	Layer	Command or Component	Output / Purpose
1	Raw data	data/ll_2018.csv to data/ll_2026.csv	Yearly traffic counter CSV files tracked with DVC.
2	Preprocessing	python main.py preprocess	Loads raw CSV files, cleans and transforms the data, engineers features, and writes data/processed/traffic_data_processed.parquet.
3	Training orchestration	python main.py train	Starts the Prefect training flow.
4	Model training	Prefect flow + scikit-learn IsolationForest	Trains the anomaly detection model on engineered traffic features.
5	Model artifact	models/isolation_forest.pkl	Stores the trained Isolation Forest, fitted scaler, feature list, and model parameters.
6	Serving	FastAPI prediction API	Loads the saved model and serves /predict, /predict/batch, /features, and /health.
7	Visualization	Streamlit dashboard	Loads processed data and the saved model to visualize traffic patterns and detected anomalies.
8	Monitoring	python main.py monitor	Runs Evidently drift monitoring against reference data.
9	Monitoring outputs	metrics.json and reports/drift/*.html	Stores machine-readable drift metrics and human-readable drift reports.

The important source modules are:

Area	Files	Purpose
CLI entrypoint	main.py	Provides commands for preprocessing, training, serving, dashboard, retraining, and monitoring.
Data loading	src/data/loader.py	Loads yearly CSV files and reads/writes parquet.
Preprocessing	src/data/preprocessor.py	Adds time fields, traffic totals, speed estimates, holiday/weekend flags, and missing-speed markers.
Feature engineering	src/features/engineer.py	Builds the model feature set and calculates the 24-hour rolling traffic average.
Model	src/models/anomaly.py	Wraps scikit-learn Isolation Forest, scaling, prediction, scoring, saving, and loading.
Training flow	src/flows/training.py	Defines Prefect tasks and flows for training and retraining.
API	src/api/app.py	FastAPI app for health checks and anomaly predictions.
Dashboard	src/dashboard/app.py	Streamlit dashboard for traffic visualization, anomaly display, and station map.
Monitoring	src/monitoring/drift.py	Evidently-based feature drift detection and HTML report generation.
Deployment	Dockerfile, docker-compose.yml, deploy/k8s/minikube.yaml	Container, local multi-service, and Minikube deployment configuration.

4. Technologies Used

The project uses the following main tools and libraries:

Technology	Role
Python 3.12	Main runtime.
pandas, NumPy, pyarrow	Data loading, transformation, and parquet storage.
scikit-learn	Isolation Forest model and StandardScaler.
joblib	Model serialization to <code>models/isolation_forest.pkl</code> .
Fire	Command-line interface in <code>main.py</code> .
DVC	Versioning raw data, processed data, model artifacts, metrics, and pipeline stages.
Backblaze B2 / S3-compatible storage	Remote DVC storage.
Prefect	Training and retraining workflow orchestration.
MLflow	Experiment tracking, parameter logging, metric logging, and model artifact logging.
Evidently AI	Data drift detection and monitoring reports.
FastAPI, Uvicorn, Pydantic	Prediction API.
Streamlit, Plotly, Folium, streamlit-folium	Interactive dashboard and station map.
Docker, Docker Compose	Local containerized deployment.
Kubernetes / Minikube	Local Kubernetes deployment.

5. DVC Pipeline

The DVC pipeline is defined in `dvc.yaml` and contains three stages.

5.1 Prepare

```
bash python main.py preprocess
```

Inputs:

- `main.py`
- `src/data`
- `src/features`
- raw yearly CSV files from 2018 to 2026

Output:

- `data/processed/traffic_data_processed.parquet`

This stage loads all available raw data, preprocesses it, engineers features, and saves the processed parquet dataset.

5.2 Train

```
bash python main.py train
```

Inputs declared in DVC:

- `main.py`
- `src/data`

- `src/features`
- `src/flows`
- `src/models`
- `data/processed/traffic_data_processed.parquet`

Output:

- `models/isolation_forest.pkl`

The DVC graph declares the processed parquet as a dependency, so training runs after preprocessing. In the current implementation, however, the Prefect training flow reloads the raw yearly CSV files internally instead of reading the parquet file directly. This means the DVC stage ordering is correct, but the training code itself performs its own load, preprocess, and feature engineering steps.

5.3 Evaluate / Monitor

```
bash python main.py monitor
```

Inputs:

- `main.py`
- `src/data`
- `src/monitoring`
- `data/processed/traffic_data_processed.parquet`
- `models/isolation_forest.pkl`

Metric output:

- `metrics.json`

This stage runs drift monitoring. It compares historical reference data against current validation-period data and writes drift metrics to `metrics.json`. It also writes HTML drift reports under `reports/drift/`.

The current `metrics.json` reports:

- 8 monitored features
- 2 drifted features
- drift share of 0.25
- maximum drift score of about 0.138

6. Feature Engineering

The model uses eight engineered features:

Feature	Description
<code>total_vehicles</code>	Sum of vehicle type columns 1 through 10.
<code>avg_speed</code>	The raw speed bucket columns are not used directly as model features. They are compressed into the <code>avg_speed</code> feature by calculating an approximate weighted average speed from the bucket midpoints.
<code>pct_heavy_vehicles</code>	Share of heavy vehicles, based on vehicle columns 6, 7, and 8.
<code>hour</code>	Hour of day extracted from <code>aeg</code> .
<code>day_of_week</code>	Day of week extracted from <code>aeg</code> , where Monday is 0.
<code>is_weekend</code>	Boolean flag for Saturday or Sunday.

Feature	Description
is_holiday	Boolean flag for selected Estonian holidays.
rolling_avg_24h	Rolling 24-hour average of <code>total_vehicles</code> , grouped by counter or counter lane.

Missing speed values are handled during feature engineering by imputing `avg_speed` with the median for the same id and hour. If that is still missing, the global median speed is used.

7. Model Training

Training is implemented in `src/flows/training.py` as a Prefect flow named `train_traffic_model`.

The default training split is:

Split	Years
Training	2018-2024
Validation	2025
Test / held-out period	2026

The model is `IsolationForest` from `scikit-learn`, wrapped by `TrafficAnomalyDetector`.

Default model parameters:

Parameter	Value
<code>n_estimators</code>	100
<code>contamination</code>	0.01
<code>max_samples</code>	auto
<code>random_state</code>	42
<code>n_jobs</code>	-1

Training steps:

1. Load training years with `TrafficDataLoader`.
2. Preprocess raw rows with `TrafficDataPreprocessor`.
3. Engineer features with `TrafficFeatureEngineer`.
4. Repeat the same process for validation year 2025.
5. Fit a `StandardScaler` on the training feature matrix.
6. Train the Isolation Forest on scaled features.
7. Evaluate on validation data.
8. Save the model, scaler, features, and parameters to `models/isolation_forest.pkl`.
9. Log parameters, metrics, model artifact, and feature list to MLflow.

The saved model file contains both the trained Isolation Forest and the fitted scaler, so the API and dashboard can apply the same preprocessing scale at prediction time.

This is an unsupervised model, so it is not evaluated against labeled examples of "correct anomaly" and "correct normal" traffic. Instead, the model learns the normal structure of historical traffic behavior from the training years and marks observations as anomalous when they are unusual compared with that learned structure. The validation step on 2025 data reports descriptive metrics such as number of samples, number of predicted anomalies, anomaly percentage, and anomaly score distribution. The `contamination=0.01` setting tells the model to expect roughly 1% of observations to be anomalous, so a reasonable result is judged by whether the anomaly rate and score distribution look plausible, not by accuracy or F1 score.

The DVC monitoring stage also does not measure prediction correctness. It checks whether the feature distribution in newer data has drifted away from the reference period. To measure true model quality, the project would need labeled incidents or manually reviewed anomaly labels, then it could calculate supervised metrics such as precision, recall, F1 score, false positives, and false negatives.

8. Prediction API

The FastAPI service is defined in `src/api/app.py`. On startup it attempts to load `models/isolation_forest.pkl`.

Main endpoints:

Endpoint	Method	Purpose
/	GET	Swagger UI because <code>docs_url="/"</code> is configured.
/health	GET	Returns service health and whether the model is loaded.
/features	GET	Returns required model feature names.
/predict	POST	Predicts anomaly status for one observation.
/predict/batch	POST	Predicts anomaly status for a list of observations.

Example prediction payload:

```
json { "total_vehicles": 150, "avg_speed": 85.5, "pct_heavy_vehicles": 0.25, "hour": 14, "day_of_week": 2, "is_weekend": false, "is_holiday": false, "rolling_avg_24h": 145 }
```

Example response fields:

Field	Meaning
<code>anomaly_score</code>	Raw Isolation Forest score. Lower values are more anomalous.
<code>prediction</code>	-1 for anomaly, 1 for normal.
<code>is_anomaly</code>	Boolean anomaly flag.
<code>anomaly_score_normalized</code>	Convenience score used by the application.

9. Dashboard

The dashboard is implemented with Streamlit in `src/dashboard/app.py`.

It supports:

- selecting years
- filtering traffic stations
- detecting anomalies on selected data
- daily traffic volume visualization
- daily average speed visualization
- anomaly table
- Folium map of traffic stations using station mapping CSV files

The dashboard uses `data/processed/traffic_data_processed.parquet` when available and falls back to loading and processing raw yearly CSV files if needed.

10. Monitoring

Monitoring is implemented in `src/monitoring/drift.py` and exposed through:

```
bash python main.py monitor
```

The current monitoring workflow:

1. Loads the processed parquet dataset if available.
2. Uses 2018-2023 as reference data.
3. Uses 2025 as current comparison data.
4. Monitors the same eight model features.
5. Runs Evidently DataDriftPreset.
6. Saves an HTML report under `reports/drift/`.
7. Writes summary metrics to `metrics.json`.

11. How to Run the Project

11.1 Local Setup

Create and activate a Python 3.12 environment:

```
bash python3.12 -m venv .venv source .venv/bin/activate pip install -r requirements.txt
```

Pull the data from DVC:

```
bash dvc pull
```

11.2 Run the DVC Pipeline

Run all DVC stages:

```
bash dvc repro
```

After these commands, the important outputs are:

- `data/processed/traffic_data_processed.parquet`
- `models/isolation_forest.pkl`
- `metrics.json`
- `reports/drift/*.html`

11.3 Run the API Locally

```
bash python main.py predict
```

The API listens on:

```
text http://localhost:8000
```

Useful URLs:

- Swagger UI: `http://localhost:8000/`
- Health check: `http://localhost:8000/health`
- Features: `http://localhost:8000/features`

11.4 Run the Dashboard Locally

```
bash python main.py dashboard
```

The dashboard normally listens on:

```
text http://localhost:8501
```

11.5 Run with Docker Compose

Build and start all services:

```
bash docker compose build docker compose up -d
```

Services:

Service	URL
FastAPI API	http://localhost:8000
Streamlit dashboard	http://localhost:8501
MLflow	http://localhost:5000
Prefect	http://localhost:4200

Stop services:

```
bash docker compose down
```

11.6 Run on Minikube

Start Minikube:

```
bash minikube start
```

Mount the project into Minikube:

```
bash mkdir -p logs .kube-data/mlflow .kube-data/prefect minikube mount "$PWD:/mnt/mlops-project"
```

Build the image inside Minikube and deploy:

```
bash eval "$(minikube docker-env)" docker build -t mlops-project:local . kubectl apply -f deploy/k8s/minikube.yaml kubectl get pods -n mlops-project
```

Open services:

```
bash minikube service api -n mlops-project minikube service dashboard -n mlops-project minikube service mlflow -n mlops-project
```

For Prefect:

```
bash kubectl port-forward svc/prefect -n mlops-project 4200:4200
```

Then open:

```
text http://localhost:4200
```